

1. 多道、并发、并行

多道:多道程序设计技术是指在计算机内存中同时存放几道相互独立的程序,相互穿插运行,共享计算机系统资源。(与之相对应的是单道程序,即在计算机内存中只允许一个的程序运行。)

并发:把 CPU 运行时间划分成若干个时间段,再将时间段分配给各个线程执行,使得两个或多个事件在同一时间间隔内发生。

并行:指两个或者多个事件在同一时刻发生。当系统有多个 CPU 时,多线程同时进行互不抢占 CPU 资源的运行方式为并行。

2. 内核态和用户态

内核态是程序在用户态进行系统调用时启动的状态,具有更高的优先级,可以使用全部指令,可以访问内存中的**所有数据以及外部设备**,也可以自由切换 CPU 等。用户态时用户执行程序时所处的状态,仅能访问部分内存,不能使用某些特权指令,**不能访问外部设备**。若不区分两种状态,会导致用户程序有意或无意的访问其他进程的资源进而可能导致系统崩溃。

3. 操作系统系统调用实现原理及调用过程

系统调用是运行在内核态的若干指令,是操作系统提供给应用程序的接口。

当进行系统调用时主要涉及以下过程:

- ①用户程序将调用参数压入堆栈
- ②调用库的过程
- ③将系统调用的编号放在合适的位置
- ④执行 trap 指令,将用户态转换成内核态的固定地址开始执行命令
- ⑤检查系统调用的编号,发出正确的系统调用命令
- ⑥内核执行相关指令
- ⑦任务执行结束后,返回用户空间的库
- ⑧返回用户的程序
- ⑨用户程序清除堆栈

4. 操作系统的未来发展方向

虚拟化:通过虚拟机实现多个不同的操作系统同时运行在裸机上,发展更小、更快、资源隔离更可信的虚拟机管理程序。

众核化:增加处理器核心数目,无需过多考虑计算资源利用率,提高操作系统执行效率。

更大的操作地址空间:实现永久对象存储,使得操作系统允许更多进程同时共享系统资源。

嵌入式:实现更强的可扩展性,可以随着所嵌入应用的需要而构建轻量级或重量级的操作系统。

无缝数据访问:提升高速缓存、预加载以及同步机制,使得操作系统在合并多台设备数据时具有更高的响应速度。

5. 几种 I/O 控制方式及其优缺点 (不太可能考)

程序控制 I/O:输入输出完全通过 CPU 执行程序来完成, CPU 轮询设备,每次执行一次 I/O 操作。

优点:实现简单,不需要过多硬件设备

缺点:CPU 和 I/O 设备串行工作;CPU 处于忙等待,利用率低;无法检测到硬件设备故障。

中断驱动 I/O: 主机启动外设后无需忙等待, 外设就绪后向主机发送中断请求, CPU 执行 I/O 控制后返回原来的程序继续执行直至外设下一次就绪。

优点: 解决了主机和外设并行工作的问题, 充分发挥处理器使用效率, 提高了系统的实时能力。

缺点: 中断驱动以字节为单位进行 I/O 操作, 需要频繁中断 CPU。

使用 DMA 的 I/O: 外设通过 DMA 控制器向 CPU 发出 DMA 请求, CPU 响应 DMA 请求后将控制权转交给 DMA, 由 DMA 执行并在结束传输后把控制权交还给 CPU。

优点: CPU 与 I/O 设备之间传输数据块而不是单个字符, 仅在传送一个或多个数据块的开始和结束需要 CPU 的干预。

缺点: DMA 方式仍需要 CPU 的控制, DMA 控制器通常执行速度比 CPU 慢很多且一次只可以控制一台外围设备, 价格昂贵。

使用通道的 I/O: 由 CPU 通过管理程序启动通道, 通道执行 CPU 指定的 I/O 工作, 执行结束后向 CPU 发中断请求, CPU 响应中断进行后续处理。

优点: DMA 方式下, 数据的传送方向、存放数据的内存定位及数据块长度等都要由 CPU 控制, 在通道方式下, 都将由通道来控制。此外, 通道设备可以做到一个通道设备控制多台设备与内存进行数据交换。

缺点: 实现复杂, 价格昂贵。

6. 产生死锁的必要条件及解决死锁的措施

必要条件: 互斥条件、占有和等待条件, 不可抢占条件、循环等待条件

解决死锁的措施: (只答蓝色部分也可以)

1. 检测死锁并从死锁中恢复
2. 避免死锁
3. 预防死锁

检测死锁并恢复:

使用资源分配图及死锁检测算法检测死锁, 让死锁发生, 一旦检测到发生立即采取行动从死锁中恢复。

从死锁中恢复的方式:

1. 利用抢占恢复: 挂起某些死锁进程并抢占它的资源, 以便让其他进程继续推进
2. 通过杀死进程恢复: 强制撤销部分甚至全部死锁进程并剥夺这些进程的资源
3. 通过回滚恢复: 让一个或多个进程回退到足以回避死锁的地步。

避免死锁:

使用银行家算法采用预分配策略检查分配完成时系统是否处在安全状态从而避免死锁的发生。

预防死锁:

破坏互斥条件: 使用 SPooling 技术

破坏占有并等待条件: 初始时申请所有资源

破坏不可剥夺条件: 对资源进行剥夺

破坏循环等待条件: 按序申请资源

7. 抖动的概念、主要原因以及如何减少系统的抖动现象

若每执行几条指令程序就发生一次缺页中断, 那么就称这个程序发生了抖动。

主要原因:

1. 置换算法选择不当

2. 采用全局置换策略
3. 当频繁缺页导致 CPU 利用率低时，CPU 为提高多道程序度，引入新的进程参与运行，形成恶性循环。

减少策略：

1. 采用局部置换策略
2. 在 CPU 调度程序中引入工作集算法，确保驻留集足够大
3. 调整多道程序度，使产生缺页的平均时间等于系统处理进程缺页的平均时间
4. 挂起若干进程，如优先级低、占用内存大、剩余时间长的进程。

8. 针对于已经很好地支持多进程的函数库，当引入线程机制时，针对全局变量的访问是否会出现问题，如果会出现问题，进一步解释出现何种问题以及解决这些问题的方法。

出现的问题：对线程而言的全局变量并不是对整个程序也是全局的。

解决方法：

1. 额外开辟一块空间，存放线程私有全局变量
2. 引入新的库过程

另一版本：

多个线程访问同一个变量，引发竞态

解决方法：

1. 采用互斥量
2. 严格轮换
3. 屏蔽中断

9. 针对于已经很好地支持多进程的函数库，当引入线程机制时，对于已有的不可重入库会面临什么问题、如何解决？

面临的问题：原有的库不支持在当前的调用没有结束之前，进行第二次调用。

解决：

1. 重写库过程
2. 为每个过程提供包装器，同一个过程一个时间只允许一个线程执行。

10. SPooling 系统的主要组成部分及其原理

主要组成部分：

1. 磁盘：输入井、输出井
2. 内存：输入缓冲区、输出缓冲区
3. 输入进程、输出进程
4. 井管理程序

原理：

当输入设备准备输入数据时，数据由输入进程送入输入缓冲区，写入输入井；当要输出数据时，若外设未就绪，先将数据写入输出井，当外设就绪时再由输出井经输出缓冲区发送至输出设备。

11. OS 空闲块管理中成组链接法原理及空闲块的分配和回收过程

OS 空闲块管理中的成组链接法是对链接法的改进，仍以链接的方式对空闲块进行管理，但

利用空闲块存放空闲块的地址，大大减少了对磁盘的访问次数，使系统资源得以充分利用。
分配方式：**分配进程先判断超级块是否被锁住**，若是则阻塞，若不是则通过栈指针取下一块空闲块并调整栈指针-1，若当前栈中只剩最后一块空闲块，要将所指向空闲块拷贝到栈中，再将这块空闲块分配出去。

回收过程：**回收进程先判断超级块是否被锁住**，若是则阻塞，若不是则向栈中添加当前空闲块，若栈已满，则将栈内数据拷贝到当前块中，将栈清空，添加当前块号到栈中；若栈未满，则直接添加当前块号到栈中。

12. 局部性原理的解释与举例

时间局部性：当某条指令被执行或某个数据被访问后，很有可能会在不久后再次被执行或被访问。举例：循环结构代码

空间局部性：当某条指令被执行或某个数据被访问后，很有可能会在不久后执行这条指令附近的指令或这个数据附近的数据。举例：数组遍历

13. 中断与陷阱的主要区别

中断一般指外中断，指来自处理机和内存外部的中断，包括 I/O 设备发出的中断，外部信号中断（例如 ESC 按键）、时钟中断、调试程序中的断点引起的调试中断。

陷阱一般指内中断：指来自处理及和内存内部的中断，包括与缺页、算术操作有误、用户程序执行特权指令、使用户态切换到内核态时发生的中断。

14. 比较进程和线程的异同点

①调度方面：在传统的操作系统中，进程是拥有资源和独立调度的基本单位，在引入了线程的现代操作系统中，进程是资源分配的基本单位，线程是调度的基本单位。

②拥有资源：无论何种操作系统，进程都是资源分配的基本单位，线程（除一部分必须外）不拥有资源，但线程可调度所隶属进程的资源

③并发性：同一程序不同进程可并发，同一进程不同线程可并发

④系统开销：创建撤销进程的开销原大于创建撤销线程的开销

⑤地址空间：不同进程各自拥有不同的地址空间，而同一进程多个线程共用所属进程的地址空间

⑥通信方面：进程间通信必须遵循系统提供的进程间通信机制，而线程间由于共用地址空间，可直接进行通信，比进程方便

15. 在 web 服务器中，进程模型和线程模型的优劣

多线程模型下，使用分派线程获得工作请求并且把工作请求分派给工作线程，每个工作线程从分派程序接收请求，检查 web 高速缓冲中是否存在所需页面，如果存在则直接返回，阻塞等待下一个请求；如果不存在则工作线程从磁盘调入该页面，将该页面返回客户机，阻塞等待下一个请求。

若使用进程模型，还是仿照上述设计思想，设计分派进程和分派进程，但此时，进程的创建撤销的开销以及通信开销要远大于线程。

16. 内核编译的过程

- ①配置编译环境
- ②下载内核文件 解压
- ③修改相关配置
- ④编译内核 make
- ⑤安装内核模块 sudo make modules_install
- ⑥安装内核 sudo make install
- ⑦重启

17. 类 Unix 操作系统

Linux
MacOs
FreeBSD
OpenBSD
MINIX

18. Linux 操作系统

Ubuntu
CentOs
Debian
RedHat
Fedora

概念

1. 局部性：时间局部性、空间局部性
2. **并发（不是并行）和共享**是现代操作系统的两大最基本特征
3. 进程调度的两种基本方式：抢占式，非抢占式
4. 页表表目的主要内容：页号和帧号

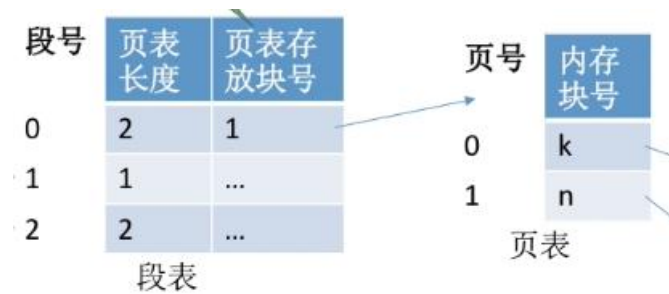
页号	内存块号
0	a
1	b
2	c

基本分页存储管理的页表

（拓展：段表表目：段号、段长、基址；

段号	段长	基址
0	7K	80K
1	3K	120K
2	6K	40K

段页式存储：段号、页表长度、页表存放块号



);

5. 在操作系统中不可中断执行的操作称为：原子操作
6. 操作系统的基本功能包括：**进程管理、存储管理、文件管理、设备管理和作业管理**
7. 操作系统是帮助管理计算机资源的一整套程序
8. 进程的三种最基本的运行状态：**运行、就绪和阻塞**
9. 原语：SUSPEND（挂起原语） ACTIVE（激活原语） BLOCK（阻塞原语） WAKEUP（唤醒原语）
- 创建/激活：把刚创建还未执行的进程设置未就绪态
- 唤醒：把阻塞态的进程设置就绪态
- 挂起：把就绪态的进程设置为阻塞态
- 阻塞：把运行态的进程设置为阻塞态
- 撤销
10. 操作系统中交互性最强的是**分时系统**
11. CPU 输入数据的速度远远高于打印机打印速度，为了解决这一矛盾，可采用**缓冲技术**
12. **缓冲技术**是操作系统中采用的以空间换时间的技术
13. **设备独立性**：应用程序独立于具体使用的物理设备
14. 虚拟存储管理系统的基础是程序的**局部性理论**
15. 文件的结构就是文件的组织形式，从用户的观点出发所看到的文件组织形式，称为文件的**逻辑结构**；从实现的观点出发，文件在外存上的存放组织形式，称为文件的**物理结构**。
16. 分时系统的特征：**多路性、交互性、独占性**
17. **顺序结构**不利于文件长度的动态增长
18. 最不利于直接存取的文件存储结构：**链接结构**
19. **虚拟设备**可以通过硬件和软件的扩充，把原来独立的设备改造成能为若干用户共享的设备
20. **原语**：由若干条指令组成的，用于完成一定功能的一个过程，具有不可分割性。
21. **竞争条件**：两个或多个进程读写某些共享数据，最后的结果取决于进程运行的精确时序，称为竞争条件。
22. **临界区**：对共享内存进行访问的程序片段称作临界区域。
23. **互斥**：以某种手段确保当一个进程在使用一个共享变量或文件时，其他进程不能做同样的操作。
24. 进程通信的原语：发送原语 SEND，接收原语 RECEIVE
25. 进程控制的原语：创建原语，撤销原语，阻塞原语，唤醒原语